# Directed Low-Diameter Decomposition for Negative-Weight Shortest Path

Jaehyun Koo

# Shortest Path Problem

Given a graph $G = (V, E)$, source $s \in V$ and a weight function $w : E \to \mathbb{Z}$, compute the length of shortest path from $s$ to every other vertices.

**If weights are nonnegative:**

- Sequentially, Dijkstra works in $O(m + n \log n)$ time.
- In parallel, recent result attains $O(n^{1/2+o(1)})$ span and $\tilde{O}(m)$ work.

# Shortest Path Problem

Given a graph $G = (V, E)$, source $s \in V$ and a weight function $w : E \to \mathbb{Z}$, compute the length of shortest path from $s$ to every other vertices.

**If weights can be negative:**

- Sequentially, Bellman-Ford works in $O(nm)$ time.
- Until 2022, fastest sequential algorithm required $\Omega(m\sqrt{n})$ time.
- Recent results based on directed LDD gives near-tight reduction to nonnegative case!
  - Robust on parallel and distributed computation
  - While preparing the talk, the algorithm was derandomized

# Roadmap

- Define Directed Low-Diameter Decomposition (Dir-LDD)
- Explain the efficient reduction from Single-Source Shortest Path (SSSP) problem to Dir-LDD
- Explain how to compute Dir-LDD quickly

**End Goal**: Persuade that SSSP doesn't get too difficult even with negative integer weights.

**Computation Model**: All algorithms are equipped with **nonnegative SSSP oracle** that takes $O(\log n)$ span and $O(n + m)$ work (to read input).

# Low-Diameter Decomposition in Undirected Graphs

- Remove **small number of edges**
- Then all components in the remaining graph has $O(\beta)$ diameter.
- Always possible with $m \log n / \beta$ edges.

Of course, LDD can be generalized to directed graphs in different ways. So we need to settle in a specific definition.

# Low-Diameter Decomposition in Directed Graphs

- Remove **small number of edges**
- Then all **strongly connected components** in the remaining graph has $O(\beta)$ diameter.
- Always possible with $m \log^2 n/\beta$ edges.

In next slide, we will present a formal defn that considers the weighted case.

# Low-Diameter Decomposition in Directed Graphs

There is a randomized algorithm $LDD(G, D)$ for any **nonnegatively** weighted graph $G$ and positive integer $D \geq 1$, which returns a set of edge $E_{rem} \subseteq E$ such that:

- Each **strongly connected components** in the remaining graph $(V, E \setminus E_{rem})$ has weak diameter $D$.
  - Forall $u, v$ in the same SCC, $dist_{G \setminus E_{rem}}(u, v) \leq D$ and $dist_{G \setminus E_{rem}}(v, u) \leq D$.
  - By weak diameter, it means that the path can use vertices outside of the SCC.
- $\forall e \in E, \mathbb{P}(e \in E_{rem}) = O(\frac{w(e) \log^2 n}{D} + \frac{1}{\text{poly}(n)})$.
- Span $\tilde{O}(1)$, work $\tilde{O}(n + m)$ under Dijkstra oracle.

# Low-Diameter Decomposition in Directed Graphs

Instead of returning the removed edges $E_{rem}$, we can instead return the list of SCC with its topological ordering, and recover $E_{rem}$ by removing edges that violates the ordering. This observation yields a more concise but equivalent definition.

# Low-Diameter Decomposition in Directed Graphs

There is a randomized algorithm $LDD(G, D)$ for any nonnegatively weighted graph $G$ and positive integer $D \geq 1$, which returns a partition $(S_1, \ldots, S_k)$ of $V$ such that:

- For any $u, v \in S_i$, $dist_G(u, v) \leq D$.
  - Each $S_i$ should be strongly connected.
  - $dist_{G \setminus E_{rem}}(u, v) = dist_{G[S_i]}(u, v)$, so the *weak diameter* thing doesn't matter.
- $\forall e \in E, \mathbb{P}(REM(e)) = O(\frac{w(e) \log^2 n}{d} + \frac{1}{\text{poly}(n)})$, where $REM((u, v)) \equiv u \in S_i, v \in S_j$ for $i > j$.
- Span $\tilde{O}(1)$, work $\tilde{O}(n + m)$ under Dijkstra oracle.

The set of SCC in both definitions are identical. However, the previous definition of LDD may remove more edges.

# Milepost

- Define Directed Low-Diameter Decomposition (Dir-LDD)
- Explain the efficient reduction from Single-Source Shortest Path (SSSP) problem to Dir-LDD $\leftarrow$ **you are here**
- Explain how to compute Dir-LDD quickly

# Restricted SSSP

We consider the following easier version of single-source shortest path, referred as Restricted SSSP.

$SSSP_R$: Given a weighted directed graph $G$ with the following restrictions:

- All edge weights are an integer in range $[-1, n]$
- For every directed cycle $C$ in $G$, $\sum_{e \in C} w(e) \geq |C|$.

Compute the SSSP from the source $s \in V$ w.h.p. (failures are allowed with prob $1/n$).

**Claim (won't prove, informal).** If you can solve $SSSP_R$, you can just repeat them for $O(\log(nW))$ time to obtain an algorithm that either finds a SSSP or returns a negative cycle. The algorithm never fails, but can be slow with prob $n^{-10}$.

# Intuition for potential function

Suppose that you are given a SCC $(S_1, S_2, \ldots, S_k)$ with their topological orders. If there is no edges of weight $-1$ connecting the same SCC, a single call is sufficient. How?

**Idea:** For each edge connecting from $S_i \to S_j$, add a weight $n(j-i)$. Then there are no negative weight edges, and all path between $S_a \to S_b$ has its weight increased by $n(b-a)$.

$n$ can be replaced to $1$, but we'll stick to this.

# Intuition for potential function

This idea can be generalized. Suppose that we have a function $\phi : V \to \mathbb{Z}$ such that $w(u, v) + \phi(u) - \phi(v) \geq 0$ for all $(u, v)$. Then:

- If we increase all edge weight by $\phi(u) - \phi(v)$, every edges have nonnegative weight, and every path from $a \to b$ has its weight increased by $\phi(a) - \phi(b)$.
- We run $SSSP_{\geq 0}$ from the source, and subtract the answer by $\phi(s) - \phi(t)$ for each $t \in V$.

In the previous case, $\phi(v) = -n \cdot (\text{topological index of SCC})$.

Such functions are called as potential function.

# Shortest paths are potentials

If you have a potential function, SSSP can be solved with single oracle call.

**Observation.** For any vertex $x \in V$, $dist(x, *)$ is a potential function.

**Proof.** $w(u, v) + dist(x, u) \geq dist(x, v)$

In some sense potential function is the end result we desire, just that it's slightly relaxed.

# Shortest paths are potentials

Throughout the rest of the talk, we use the following way of computing the potential function:

- Create a dummy vertex $s'$, and add a directed edge of $s' \to v$ with weight $0$ for all $v \in V$. Then we use $dist(s', s)$ as a potential function.

We call this specific method as a *potential finding process*. The dummy vertex $s'$ is defined per the potential finding process.

Good thing about this process is that:

- It always works regardless of the connectivity of the graph.
- $dist(s', v) \leq 0$ for all $v$.

# Warmup: Potentials in $\tilde{O}(m\sqrt{n})$ work, $\tilde{O}(1)$ span

Let $G_{\geq 0}$ be a graph where all edges of weight $-1$ is replaced to $0$, and run the LDD with $D = \sqrt{n}$. We want to show:

- If we run a potential finding process over $G$, the expected number of removed (in LDD) edges for all shortest path from $s'$ are at most $\tilde{O}(\sqrt{n})$.
- If we run a potential finding process over $G[S_i]$, there exists a shortest path from $s'$ with at most $D$ edges.

# Warmup: Potentials in $\tilde{O}(m\sqrt{n})$ work, $\tilde{O}(1)$ span

**(Pt 1)** If we run a potential finding process over $G$, the expected number of removed (in LDD) edges for all shortest path from $s'$ are at most $\tilde{O}(\sqrt{n})$.

- By construction, we have $dist_G(s', v) \leq 0$. Hence for all shortest paths in $G$, its projection in $G'$ has length at most $n$.
- From the defn. of LDD, we have that $\mathbb{P}(e \in E_{rem}) = O(\frac{w(e)\log^2 n}{\sqrt{n}} + \frac{1}{\text{poly}(n)}) = \tilde{O}(w(e)/\sqrt{n})$.
- Applying linearity of expectation for each edge in the shortest path, the expected number of removed edges is at most $\tilde{O}(\sqrt{n})$.

# Warmup: Potentials in $\tilde{O}(m\sqrt{n})$ work, $\tilde{O}(1)$ span

**(Pt 2)** If we run a potential finding process over $G[S_i]$, there exists a shortest path from $s'$ with at most $D$ edges. ($D \geq 1$)

- Suppose that the path $s' \to v$ has $D + 1$ edges.
- Let $u$ be the next vertex encountered after leaving $s'$.
- By construction, path $u \to v$ has weight at most $0$ in $G$.
- By LDD, there's a path $v \to u$ with weight at most $D$ in $G_{\geq 0}$, and hence also in $G$.
- If we concatenate them, we obtain a cycle with at least $D + 1$ edges and weight at most $D$ in $G$, contradicting the mean-weight assumption of $SSSP_R$.

# Warmup: Potentials in $\tilde{O}(m\sqrt{n})$ work, $\tilde{O}(1)$ span

The potential finding process can assume that the path of interest only have $k = \tilde{O}(\sqrt{n})$ negative edges. With this restriction you can compute SSSP from any source.

Run dynamic programming (-ish) process. Create $k+1$ copies of the graph $G + s'$, where each vertex $v$ is duplicated to $k+1$ copies labeled $(v,0),(v,1),\ldots,(v,k)$. Then:

- For each nonnegative edge $(u,v)$ add an edge from $(u,i) \to (v,i)$
- For each weight $-w_i$ edge $(u,v)$ add an edge from $(u,i) \to (v,i+1)$ with weight $\infty - w_i$.

Use SSSP oracle in the graph. We can see that $\phi(v) = \min_{0 \le i \le k}(dist((s',0),(v,i)) - i \cdot \infty)$.

# Warmup: Potentials in $\tilde{O}(m\sqrt{n})$ work, $\tilde{O}(1)$ span

So, we can compute a potential function that:

- (a) fixes intra-cluster edges (Pt 2 + DP)
- (b) fixes inter-cluster edges that are removed. (Pt 1 + DP)
- (c) fixes inter-cluster edges that are not removed (from intuition part)

You can't add up and call it a day, since fixing some edges may ruin the other edges. But at least (a) + (c) can be combined, so we only have negative edges in $E_{rem}$.

For any path, $\mathbb{E}[E_{rem} \cap P] \leq \tilde{O}(\sqrt{n})$, hence computing the SSSP over this graph will succeed with constant probability by Markov inequality that can be boosted to WHP.

**Pants On Fire!** Actually it won't succeed with constant probability, because we need upper bounds for *every vertex*. Later we will see a workaround.

# Potentials in $\tilde{O}(m)$ work, $\tilde{O}(1)$ span

In general, the work necessary to compute each potentials are:

- (a) fixes intra-cluster edges -- $O(D)$ edges in path
- (b) fixes inter-cluster edges that are removed. -- $\tilde{O}(n/D)$ negative edges in path
- (c) fixes inter-cluster edges that are not removed -- immediate from SCC

**Optimization Idea**: Start from $D = O(n)$. Then case (b) and (c) is easy. For case (a), recurse in each intra-cluster with to compute the potential.

We will present a very succinct take of this high-level idea (SOSA'25) that:

- fixes the previous inaccuracy in $\tilde{O}(m\sqrt{n})$ nicely
- is immediately parallelizable

# Potentials in $\tilde{O}(m)$ work, $\tilde{O}(1)$ span

**Parallel Take:** Ignore case (b) (!!). Just repeat the following:

- For $D = 1, 2, 4, \ldots$, compute the LDD of $G_{\geq 0}$ for $\beta = D$.
- For each cluster, compute the potential for intra-cluster edge with $k = O(\log^2 n)$, using the potential of $D/2$.
- Compute inter-cluster edge conforming to the SCC
- Combine and declare it as a potential for $D$.

**Fixing the intra-cluster edges.** By LDD guarantee of $D$, the cluster has diameter at most $D$ per $G_{\geq 0}$. By LDD guarantee of $D/2$, there are at most $O(D/(D/2)\log^2 n)$ edges that are not fixed by the potential of $D/2$. We have an $O(\log^2 n)$ expectation for each vertex, which we know is not enough.

# Potentials in $\tilde{O}(m)$ work, $\tilde{O}(1)$ span

**Fixing the expectation problem.** Our potential finding procedure never underestimates the distance. Hence we can run it for $O(\log n)$ times and take the elementwise minimum!

**Final Algorithm:** We need $O(\log n)$ potentials derived from independent randomization of LDD. So we need to repeat the above procedure $O(\log n)$ times. If you do that until $D = poly(n)$, the LDD would not return more than one SCC whp and we are done.

**Log Counts**: $1$ for scaling, $1$ for doubling, and:

- $O(\log n)$ LDD calls (for whp), each is $O(\log^3 n)$ SSSP call I think.
- $O(\log^2 n)$ SSSP calls (for whp) of graph with $O(\log^2 n)$ times bigger size.

$O(\log^6 n)$ times slower than Dijkstra.

# Milepost

- Define Directed Low-Diameter Decomposition (Dir-LDD)
- Explain the efficient reduction from Single-Source Shortest Path (SSSP) problem to Dir-LDD
- Explain how to compute Dir-LDD quickly $\leftarrow$ **you are here**

# Directed Low-Diameter Decomposition

- Each **strongly connected components** in the remaining graph $(V, E \setminus E_{rem})$ has **weak** diameter $D$.
  - For any $u, v$ in the same SCC, $dist_G(u, v) \leq D$ and $dist_G(v, u) \leq D$.
  - That path can use vertices outside of the SCC, but cannot use edges in $E_{rem}$.
- $\forall e \in E, \mathbb{P}(e \in E_{rem}) = O(\frac{w(e)\log^2 n}{D} + \frac{1}{\text{poly}(n)})$.

**Roadmap:** We use divide-and-conquer in the graph, aiming *smaller subproblem* and *small number of removed edges*.

In general, if one divides the graph into $S, V \setminus S$ and compute $LDD(S), LDD(V \setminus S)$, one can decide to either remove all edges from $S \to V \setminus S$, or $V \setminus S \to S$ and obtain the solution. The issue is to compute the right $S$ so that there are not much removed edges.

# Ball Carving - (1) Idea

We define the following:

- *out-ball* $B_G^{out}(s, R) = \{v \in V \mid dist_G(s, v) \leq R\}$
- *in-ball* $B_G^{in}(s, R) = \{v \in V \mid dist_G(v, s) \leq R\}$
- *out-ball-cut* $B_G^{out}(s, R) = \{(u, v) \in E \mid u \in B_G^{out}(s, R), v \notin B_G^{out}(s, R)\}$
- *in-ball-cut* $B_G^{in}(s, R) = \{(u, v) \in E \mid u \notin B_G^{in}(s, R), v \in B_G^{in}(s, R)\}$

**Idea:** Take any vertex $v$ and a random $R \in Geom(20 \log n/D)$. (assume $R \leq D/2$).

- If the *out-ball* is smaller than $0.9n$, remove the out-ball and out-ball-cut.
- If the *in-ball* is smaller than $0.9n$, remove the in-ball and in-ball-cut.
- What if both are not possible?

# Ball Carving - (1) Idea

**Idea:** Take any arbitrary vertex $v$ and a random radius $R \in Geom(20 \log n/D)$. (assume $R \leq D/2$).

- If the *out-ball* is smaller than $0.9n$, remove the out-ball and out-ball-cut.
- If the *in-ball* is smaller than $0.9n$, remove the in-ball and in-ball-cut.

*Lemma*. If there is no $v$ such that $|B_G^{in}(v, R)| \leq 0.9n$ and $|B_G^{out}(v, R)| \leq 0.9n$, then the graph have weak diameter $D$.

*Proof:* For any vertex $u, v$, consider any vertex $w \in B_G^{out}(u, R) \cap B_G^{in}(v, R)$ which exists. $dist(u, v) \leq dist(u, w) + dist(w, v) \leq 2R \leq D$.

# Ball Carving - (2) Algorithm

- For each vertex $v$, compute $|B_G^{in}(v, D)|$ and $|B_G^{out}(v, D)|$. If one of them are at least $0.9n$, call $v$ good.
- Take any arbitrary good vertex $v$ and a random radius $R \in Geom(20 \log n/D)$. (assume $R \leq D/2$). Since it's good, either its out-ball or in-ball per the radius is at most $0.9n$. Remove the out-ball and out-ball-cut (or vice versa), and recurse on the out-ball.
- Repeat until you run out of good vertex. Cluster the remaining non-good vertices into a single set.

# Ball Carving - (3) Proof

- We know that each SCC has weak diameter $D$.
- For the ball carving, we use the notion of ball "expanding": For each vertex $v$, we toss a biased coin with $p = \frac{20 \log n}{D}$ chance of head. Increase the radius by $1$ for each tail toss, and terminate if we got a head.
- For each edge $e = (u \to v)$, consider the first moment where either $u$ and $v$ belongs to a ball. (If that doesn't exist, $e$ is not cut).
- WLOG assume $u$ first got into the out-ball. $e$ belongs to the out-ball-cut if the coin toss reached a head before $v$ also got into the out-ball. This happens w.p $\leq O(\frac{20w(e) \log n}{D})$.
- If both belongs to a ball, we repeat this on smaller subproblem, adding an $O(\log n)$ multiplicative factor.

# Ball Carving - (4) Implementation

- **Finding Good Vertices:** Boils down to computing the size of each balls. Naively this takes $O(n^2)$ time, but the size parameter here can be anything in range $(0.5n, 0.99n)$. So one can sample $O(\log n)$ random vertex to estimate the size of the balls. This takes $\tilde{O}(m+n)$ work and $\tilde{O}(1)$ span.
- **Carving out the balls:** Sequentially, one can simply repeat the SSSP to find out the actual balls and eliminate them. On the other hand, this procedure is somewhat sequential to apply for parallel model.

# Ball Carving - (4) Implementation

**Parallel Ball Carving.**

- Observe that you don't actually need to "remove" the carved ball. One can carve ball $B_G^{in}(v_1, R_1) \cup B_G^{in}(v_2, R_2)$ even if $v_2 \in B_G^{in}(v_1, R_1)$. This doesn't hurt the correctness or probability guarantee.
- This removes the "sequential" aspect of the ball carving.
- Binary search for the smallest prefix $j$ such that $|\cup_{j \leq i} B_G^{in/out}(v_j, R_j)| > 0.05n$. That set has size at most $0.95n$, so you can recursively solve each subproblems.
- If such prefix does not exist for both in/out, the entire component except the ball is strongly connected.

# References


- A Simple Parallel Algorithm with Near-Linear Work for Negative-Weight Single-Source Shortest Path, SOSA'25
- Notes on LDD by Danupon Nanongkai
- Parallel, Distributed, and Quantum Exact Single-Source Shortest Paths with Negative Edge Weights, ESA'24